```
import numpy as np
import pandas as pd

np.random.seed(42)
n_students = 300

# Base latent aptitude distribution
aptitude = np.random.normal(loc=0.0, scale=1.0, size=n_students)

student_ids = [f"STU_{i:03d}" for i in range(1, n_students + 1)]

prior_gpa = np.clip(
    np.round(6.8 + 1.25 * aptitude + np.random.normal(0, 0.45,
n_students), 2),
    4.0, 10.0
)

study_hours = np.clip(
    np.round(15 + 4.5 * aptitude + np.random.normal(0, 2.0,
n_students)),
    3, 35
).astype(int)

# Subject parameters: (mean_att, mean_marks, std_att, std_marks)
subject_params = {
    'DBMS': {'att_base': 77, 'mark_base': 64, 'difficulty': 1.0},
    'ML':   {'att_base': 73, 'mark_base': 58, 'difficulty': 1.2},
    'AI':   {'att_base': 75, 'mark_base': 61, 'difficulty': 1.1},
    'CV':   {'att_base': 71, 'mark_base': 55, 'difficulty': 1.3}
}

data = {
    'Student_ID': student_ids,
    'Prior_GPA': prior_gpa,
    'Study_Hours_Week': study_hours
}

target_cols = {}

for sub, cfg in subject_params.items():
    # Generate Attendance (35% to 100%)
    att = np.clip(
        np.round(cfg['att_base'] + 10 * aptitude + np.random.normal(0,
7.5, n_students)),
        35, 100
    ).astype(int)

    # Generate Midterm Marks (0 to 100)
    marks = np.clip(
        np.round(cfg['mark_base'] + 14 * aptitude +
```

```
np.random.normal(0, 8.5, n_students)),
        12, 98
    ).astype(int)

    # Generate Assignments Submitted (0 to 5)
    assign = np.clip(
        np.round(3.6 + 0.9 * aptitude + np.random.normal(0, 0.6,
n_students)),
        0, 5
    ).astype(int)

    data[f'Att_{sub}'] = att
    data[f'Marks_{sub}'] = marks
    data[f'Assign_{sub}'] = assign

    # Subject-Specific Risk Function
    # Critical risk triggers: Attendance < 75%, Marks < 40,
Assignments <= 2
    att_deficit = np.maximum(0, 75 - att)
    mark_deficit = np.maximum(0, 40 - marks)

    logit = (
        0.18 * mark_deficit
        + 0.12 * att_deficit
        - 0.60 * assign
        - 0.40 * (prior_gpa - 6.5)
        - 0.05 * (study_hours - 15)
        - 1.8
    )
    prob = 1.0 / (1.0 + np.exp(-logit))
    target_cols[f'Risk_{sub}'] = (prob >= 0.5).astype(int)

# Combine features and 4 target labels
df = pd.DataFrame({**data, **target_cols})

# Save to CSV
df.to_csv("student_subject_risk_300.csv", index=False)
print("Saved to student_subject_risk_300.csv")
print("\nRisk distribution per subject:")
for col in ['Risk_DBMS', 'Risk_ML', 'Risk_AI', 'Risk_CV']:
    pct = df[col].mean() * 100
    print(f"  {col}: {df[col].sum()}/{n_students} at-risk ({pct:.1f}
%)")
```

```
Saved to student_subject_risk_300.csv

Risk distribution per subject:
  Risk_DBMS: 15/300 at-risk (5.0%)
  Risk_ML: 39/300 at-risk (13.0%)
```

```
  Risk_AI: 26/300 at-risk (8.7%)
  Risk_CV: 39/300 at-risk (13.0%)
```

```python
import numpy as np
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    confusion_matrix,
    classification_report
)

# 1. Load the existing CSV file
csv_filename = "student_subject_risk_300.csv"  # Update with your file path if different
df = pd.read_csv(csv_filename)

subjects = ['DBMS', 'ML', 'AI', 'CV']
results = []
models = {}

print("=" * 70)
print("LOGISTIC REGRESSION EVALUATION REPORT (PER SUBJECT)")
print("=" * 70)

for sub in subjects:
    # Feature columns specific to this course plus general background features
    feature_cols = ['Prior_GPA', 'Study_Hours_Week', f'Att_{sub}', f'Marks_{sub}', f'Assign_{sub}']
    target_col = f'Risk_{sub}'

    X = df[feature_cols]
    y = df[target_col]

    # 80/20 Stratified Split to preserve at-risk class balance
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.20, random_state=42, stratify=y
    )

    # Feature Standardization (ensures stable convergence for gradient/solver)
    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)
```

```
    X_test_scaled = scaler.transform(X_test)

    # Train Logistic Regression
    clf = LogisticRegression(solver='lbfgs', max_iter=1000,
random_state=42)
    clf.fit(X_train_scaled, y_train)
    models[sub] = (clf, scaler)

    # Model Predictions & Probabilities
    y_pred = clf.predict(X_test_scaled)
    y_prob = clf.predict_proba(X_test_scaled)[:, 1]

    # Compute Metrics
    acc = accuracy_score(y_test, y_pred)
    prec = precision_score(y_test, y_pred, zero_division=0)
    rec = recall_score(y_test, y_pred, zero_division=0)
    f1 = f1_score(y_test, y_pred, zero_division=0)
    auc = roc_auc_score(y_test, y_prob)
    cm = confusion_matrix(y_test, y_pred)

    results.append({
        'Subject': sub,
        'Accuracy': f"{acc * 100:.2f}%",
        'Precision': f"{prec * 100:.2f}%",
        'Recall': f"{rec * 100:.2f}%",
        'F1-Score': round(f1, 4),
        'ROC-AUC': round(auc, 4),
        'Confusion Matrix [TN, FP, FN, TP]': cm.ravel().tolist()
    })

    print(f"\n--- Subject: {sub} ---")
    print(f"Confusion Matrix:\n{cm}")
    print(f"Accuracy: {acc:.4f} | Recall: {rec:.4f} | F1: {f1:.4f} |
ROC-AUC: {auc:.4f}")

# Display Summary Comparison Table
results_df = pd.DataFrame(results)
print("\n" + "=" * 70)
print("METRICS SUMMARY ACROSS ALL 4 SUBJECTS")
print("=" * 70)
print(results_df.to_string(index=False))
```

```
======================================================================
LOGISTIC REGRESSION EVALUATION REPORT (PER SUBJECT)
======================================================================

--- Subject: DBMS ---
Confusion Matrix:
[[56  1]
 [ 1  2]]
```

```
Accuracy: 0.9667 | Recall: 0.6667 | F1: 0.6667 | ROC-AUC: 0.9883

--- Subject: ML ---
Confusion Matrix:
[[52  0]
 [ 0  8]]
Accuracy: 1.0000 | Recall: 1.0000 | F1: 1.0000 | ROC-AUC: 1.0000

--- Subject: AI ---
Confusion Matrix:
[[53  2]
 [ 0  5]]
Accuracy: 0.9667 | Recall: 1.0000 | F1: 0.8333 | ROC-AUC: 1.0000

--- Subject: CV ---
Confusion Matrix:
[[52  0]
 [ 1  7]]
Accuracy: 0.9833 | Recall: 0.8750 | F1: 0.9333 | ROC-AUC: 1.0000

================================================================================
METRICS SUMMARY ACROSS ALL 4 SUBJECTS
================================================================================
Subject Accuracy Precision  Recall  F1-Score  ROC-AUC Confusion Matrix
[TN, FP, FN, TP]
   DBMS   96.67%    66.67%  66.67%    0.6667   0.9883
[56, 1, 1, 2]
     ML  100.00%   100.00% 100.00%    1.0000   1.0000
[52, 0, 0, 8]
     AI   96.67%    71.43% 100.00%    0.8333   1.0000
[53, 2, 0, 5]
     CV   98.33%   100.00%  87.50%    0.9333   1.0000
[52, 0, 1, 7]

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score,
    roc_auc_score,
    confusion_matrix
)
```

```python
# Visual styling setup
sns.set_theme(style="whitegrid", font_scale=1.0)
plt.rcParams['font.sans-serif'] = 'DejaVu Sans'

# 1. Load Dataset
df = pd.read_csv("student_subject_risk_300.csv")
subjects = ['DBMS', 'ML', 'AI', 'CV']

metrics_list = []
confusion_matrices = {}

# 2. Train and Evaluate Logistic Regression for Each Subject
for sub in subjects:
    feature_cols = ['Prior_GPA', 'Study_Hours_Week', f'Att_{sub}',
f'Marks_{sub}', f'Assign_{sub}']
    X = df[feature_cols]
    y = df[f'Risk_{sub}']

    # 80/20 Stratified Split
    X_train, X_test, y_train, y_test = train_test_split(
        X, y, test_size=0.20, random_state=42, stratify=y
    )

    # Feature Standardization
    scaler = StandardScaler()
    X_train_scaled = scaler.fit_transform(X_train)
    X_test_scaled = scaler.transform(X_test)

    # Logistic Regression Model
    lr = LogisticRegression(solver='lbfgs', max_iter=1000,
random_state=42)
    lr.fit(X_train_scaled, y_train)

    # Predictions & Probabilities
    y_pred = lr.predict(X_test_scaled)
    y_prob = lr.predict_proba(X_test_scaled)[:, 1]

    # Compute Metrics
    acc = accuracy_score(y_test, y_pred)
    prec = precision_score(y_test, y_pred, zero_division=0)
    rec = recall_score(y_test, y_pred, zero_division=0)
    f1 = f1_score(y_test, y_pred, zero_division=0)
    auc = roc_auc_score(y_test, y_prob)

    metrics_list.append({
        'Subject': sub,
        'Accuracy': acc,
        'Precision': prec,
        'Recall': rec,
```

```
        'F1-Score': f1,
        'ROC-AUC': auc
    })

    # Store Confusion Matrix
    confusion_matrices[sub] = confusion_matrix(y_test, y_pred)

# Display tabular summary in terminal
df_metrics = pd.DataFrame(metrics_list)
print("=" * 65)
print("LOGISTIC REGRESSION PERFORMANCE SUMMARY")
print("=" * 65)
print(df_metrics.to_string(index=False))

#
# PLOT 1: Performance Metrics Across All Subjects
#
plt.figure(figsize=(10, 5.5))
df_melted = df_metrics.melt(id_vars='Subject', var_name='Metric',
value_name='Score')

ax1 = sns.barplot(
    data=df_melted,
    x='Subject',
    y='Score',
    hue='Metric',
    palette='Blues_d'
)

# Add exact score labels on top of each bar
for p in ax1.patches:
    val = p.get_height()
    if val > 0:
        ax1.annotate(f"{val:.2f}",
                     (p.get_x() + p.get_width() / 2., val),
                     ha='center', va='bottom', fontsize=8,
                     xytext=(0, 2), textcoords='offset points')

plt.title('Logistic Regression Performance Metrics Across Subjects',
fontsize=13, fontweight='bold', pad=12)
plt.ylim(0.5, 1.05)
plt.ylabel('Score (0.0 - 1.0)', fontsize=11)
plt.xlabel('Subject', fontsize=11)
plt.legend(loc='lower right', frameon=True)
plt.tight_layout()
plt.show()

#
```

```
# ============================================================================
# PLOT 2: Confusion Matrix Heatmaps (2x2 Grid)
# ============================================================================
fig, axes = plt.subplots(2, 2, figsize=(10, 8))

for idx, sub in enumerate(subjects):
    ax = axes[idx // 2, idx % 2]
    cm = confusion_matrices[sub]
    sns.heatmap(
        cm,
        annot=True,
        fmt='d',
        cmap='Blues',
        cbar=False,
        annot_kws={'size': 13, 'weight': 'bold'},
        ax=ax
    )

    ax.set_title(f'Confusion Matrix: {sub}', fontsize=12,
fontweight='bold')
    ax.set_xlabel('Predicted Label (0 = Safe, 1 = At-Risk)',
fontsize=10)
    ax.set_ylabel('Actual Label (0 = Safe, 1 = At-Risk)', fontsize=10)

plt.suptitle('Logistic Regression Classification Outcomes',
fontsize=15, fontweight='bold', y=0.98)
plt.tight_layout()
plt.show()

# ============================================================================
# PLOT 3: 2D Decision Boundary (Attendance vs. Marks in ML)
# ============================================================================
sub_case = 'ML'
X_2d = df[[f'Att_{sub_case}', f'Marks_{sub_case}']].values
y_2d = df[f'Risk_{sub_case}'].values

scaler_2d = StandardScaler()
X_2d_scaled = scaler_2d.fit_transform(X_2d)

# Fit 2D model to project the linear separating line
lr_2d = LogisticRegression().fit(X_2d_scaled, y_2d)

# Create 2D coordinate mesh
x_min, x_max = X_2d_scaled[:, 0].min() - 0.5, X_2d_scaled[:, 0].max()
+ 0.5
y_min, y_max = X_2d_scaled[:, 1].min() - 0.5, X_2d_scaled[:, 1].max()
+ 0.5
```

```
xx, yy = np.meshgrid(np.linspace(x_min, x_max, 300),
np.linspace(y_min, y_max, 300))

grid_points = np.c_[xx.ravel(), yy.ravel()]
preds = lr_2d.predict(grid_points).reshape(xx.shape)

plt.figure(figsize=(8, 6))
plt.contourf(xx, yy, preds, alpha=0.25, cmap='coolwarm')
plt.contour(xx, yy, preds, colors='black', linewidths=1.5)

# Scatter actual student points
plt.scatter(
    X_2d_scaled[:, 0], X_2d_scaled[:, 1],
    c=y_2d, cmap='coolwarm', edgecolors='k', s=40, alpha=0.85
)

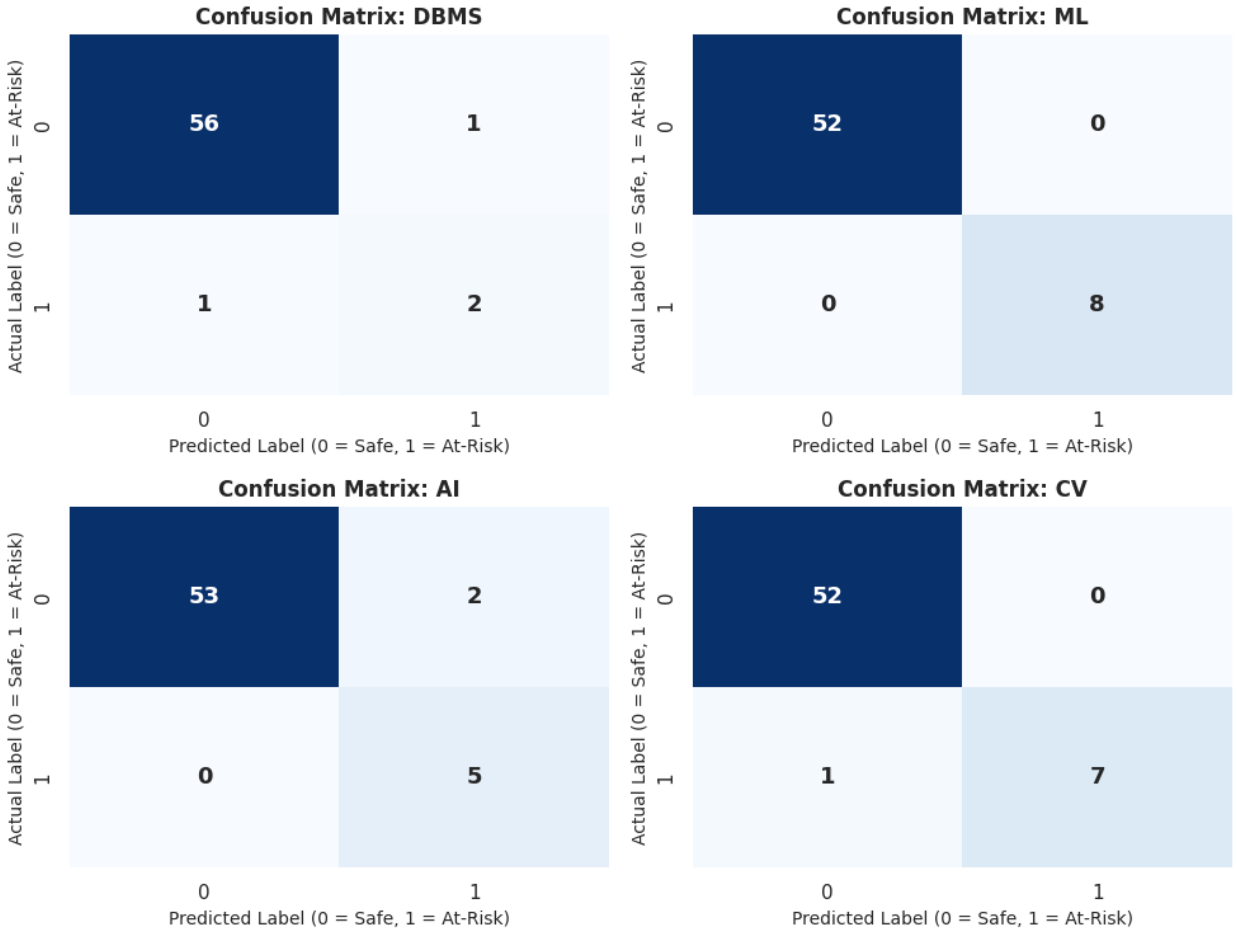
legend_elements = [
    plt.Line2D([0], [0], marker='o', color='w', label='Safe (0)',
markerfacecolor='#3B4CC0', markersize=8),
    plt.Line2D([0], [0], marker='o', color='w', label='At-Risk (1)',
markerfacecolor='#B40426', markersize=8),
    plt.Line2D([0], [0], color='black', lw=1.5, label='Linear Decision
Boundary')
]
plt.legend(handles=legend_elements, loc='upper left')

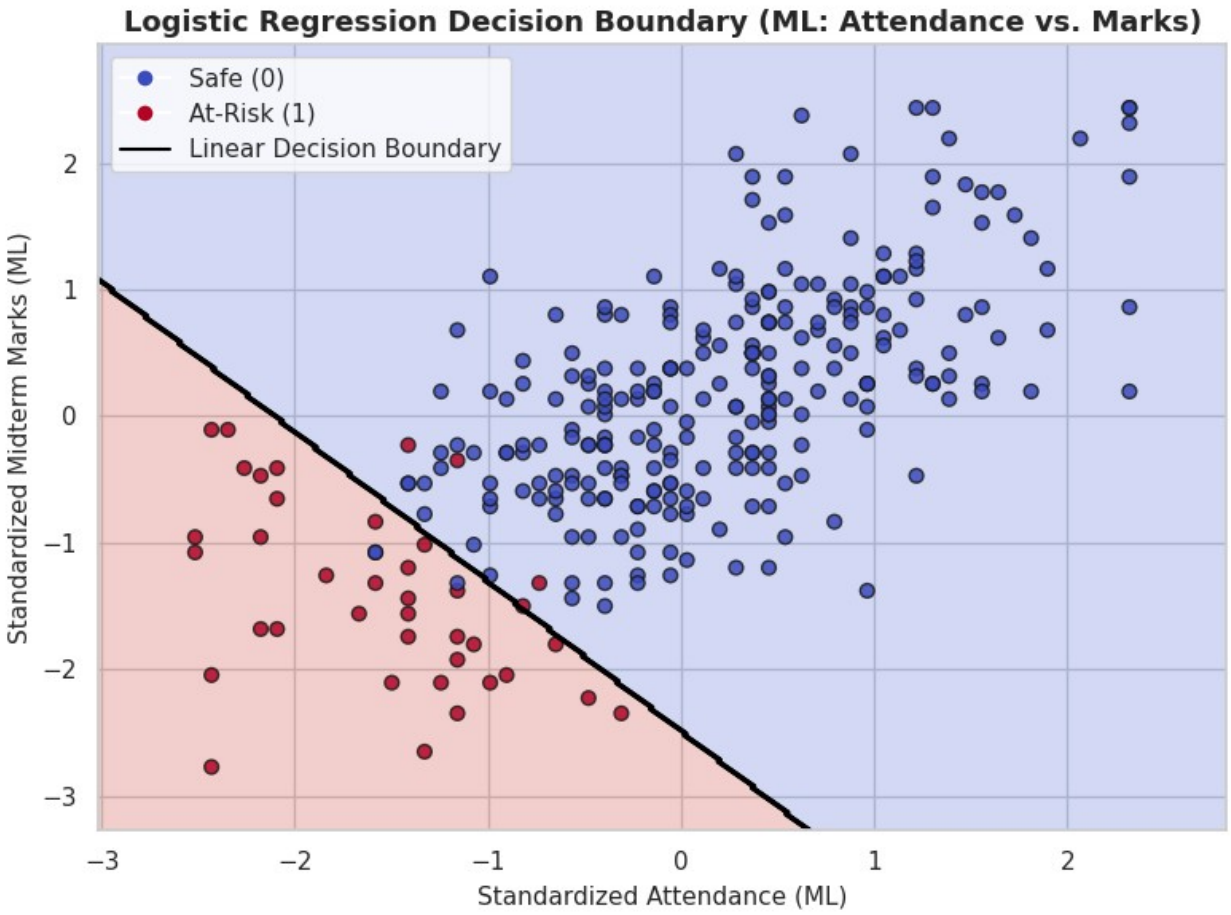
plt.title(f'Logistic Regression Decision Boundary ({sub_case}:
Attendance vs. Marks)', fontsize=13, fontweight='bold')
plt.xlabel(f'Standardized Attendance ({sub_case})', fontsize=11)
plt.ylabel(f'Standardized Midterm Marks ({sub_case})', fontsize=11)
plt.tight_layout()
plt.show()
```

```
================================================================
LOGISTIC REGRESSION PERFORMANCE SUMMARY
================================================================
Subject  Accuracy  Precision   Recall  F1-Score  ROC-AUC
   DBMS  0.966667   0.666667 0.666667  0.666667 0.988304
     ML  1.000000   1.000000 1.000000  1.000000 1.000000
     AI  0.966667   0.714286 1.000000  0.833333 1.000000
     CV  0.983333   1.000000 0.875000  0.933333 1.000000
```



Logistic Regression Classification Outcomes




Logistic Regression Decision Boundary (ML: Attendance vs. Marks)
Safe (0)
At-Risk (1)
Linear Decision Boundary
Standardized Midterm Marks (ML)
Standardized Attendance (ML)
2
1
0
−1
−2
−3
−3
−2
−1
0
1
2